12. Core 컨트랙트 구조 분석

개요

Omni 네트워크의 Core 컨트랙트는 크로스체인 메시징의 핵심 온체인 로직을 구현합니다. 주요 컴포넌트로는 OmniPortal, FeeOracle, PortalRegistry가 있으며, 이들은 함께 안전하고 효율적인 크로스체인 통신을 제공합니다.

디렉토리 구조

```
contracts/core/src/
├── xchain/ # 크로스체인 핵심 컨트랙트
| ├── OmniPortal.sol # 메인 포털 컨트랙트
| ├── FeeOracleV2.sol # 수수료 오라클
| ├── PortalRegistry.sol # 포털 레지스트리
| └── ...
├── interfaces/ # 컨트랙트 인터페이스
├── libraries/ # 공통 라이브러리
├── utils/ # 유틸리티 컨트랙트
└── ...
```

1. OmniPortal 컨트랙트

1.1 핵심 기능

OmniPortal은 Omni 네트워크의 중심 컨트랙트로, 크로스체인 메시지의 송수신을 담당합니다.

주요 상속 구조:

```
contract OmniPortal is
IOmniPortal,
IOmniPortalSys,
IOmniPortalAdmin,
OwnableUpgradeable,
PausableUpgradeable,
ReentrancyGuardUpgradeable,
OmniPortalConstants,
OmniPortalStorage
```

1.2 초기화 파라미터

```
struct InitParams {
address owner; // 컨트랙트 소유자
address feeOracle; // 수수료 오라클 주소
uint64 omniChainId; // Omni EVM 체인 ID
uint64 omniCChainId; // Omni 합의 체인 ID
uint64 xmsgMaxGasLimit; // 최대 가스 한도
uint64 xmsgMinGasLimit; // 최소 가스 한도
```

```
uint16 xmsgMaxDataSize; // 최대 데이터 크기

uint16 xreceiptMaxErrorSize; // 최대 에러 크기

uint8 xsubValsetCutoff; // 검증자 세트 컷오프

uint64 cChainXMsgOffset; // 합의 체인 메시지 오프셋

uint64 cChainXBlockOffset; // 합의 체인 블록 오프셋

uint64 valSetId; // 초기 검증자 세트 ID

XTypes.Validator[] validators; // 초기 검증자 목록

}
```

1.3 아웃바운드 xcall 함수

xcall 함수:

```
function xcall(

uint64 destChainId, // 목적지 체인 ID

uint8 conf, // 확인 레벨

address to, // 호출할 컨트랙트 주소

bytes calldata data, // 함수 호출 데이터

uint64 gasLimit // 가스 한도

) external payable
```

실행 과정:

1. 목적지 체인 지원 여부 확인
2. 가스 한도 및 데이터 크기 검증
3. 수수료 계산 및 검증

4. 메시지 오프셋 증가

5. `XMsg` 이벤트 발생

수수료 계산:

```
function feeFor(uint64 destChainId, bytes calldata data, uint64 gasLimit)

public view returns (uint256) {

return IFeeOracle(feeOracle).feeFor(destChainId, data, gasLimit);

}
```

1.4 인바운드 xsubmit 함수

xsubmit 함수:

```
function xsubmit(XTypes.Submission calldata xsub) external
```

검증 과정:

1. **기본 검증:**

- 합의 체인 ID 확인
- 메시지 존재 여부 확인
- 검증자 세트 유효성 확인

2. **쿼럼 검증:**

```
require(

Quorum.verify(

xsub.attestationRoot,
```

```
xsub.signatures,
valSet[valSetId],
valSetTotalPower[valSetId],
XSubQuorumNumerator,
XSubQuorumDenominator
),
"OmniPortal: no quorum"
);
```

3. **머클 증명 검증:**

```
require(
XBlockMerkleProof.verify(
xsub.attestationRoot,
xheader,
xmsgs,
xsub.proof,
xsub.proofFlags
),
"OmniPortal: invalid proof"
);
```

1.5 메시지 실행 로직

_exec 함수:

```
function _exec(XTypes.BlockHeader calldata xheader, XTypes.Msg calldata xmsg_) i
```

실행 단계:

1. **기본 검증:**

- 목적지 체인 ID 확인
- 메시지 오프셋 순서 확인
- 확인 레벨 일치 여부 확인

2. **시스템 콜 vs 일반 콜 구분:**

```
bool isSysCall = xmsg_.to == VirtualPortalAddress;



if (isSysCall) {

// 시스템 콜: 합의 체인에서만 허용

require(

xheader.sourceChainId == omniCChainId &&

xmsg_.sender == CChainSender &&

xmsg_.destChainId == BroadcastChainId

);

} else {

// 일반 콜: 합의 체인 제외

require(

xheader.sourceChainId != omniCChainId &&

xmsg_.sender != CChainSender
```

```
);
}
```

3. **실제 호출 실행:**

```
_xmsg = XTypes.MsgContext(sourceChainId, xmsg_.sender);
(bool success, bytes memory result, uint256 gasUsed) =
isSysCall ? _syscall(xmsg_.data) : _call(xmsg_.to, xmsg_.gasLimit, xmsg_.data);
delete _xmsg;
```

1.6 외부 컨트랙트 호출

_call 함수:

```
function _call(address to, uint256 gasLimit, bytes calldata data)
internal returns (bool, bytes memory, uint256) {
uint256 gasLeftBefore = gasleft();
// 안전한 외부 호출 (큰 반환 데이터 방지)
(bool success, bytes memory result) = to.excessivelySafeCall({
_gas: gasLimit,
_value: 0,
_maxCopy: xreceiptMaxErrorSize,
_calldata: data
});
uint256 gasLeftAfter = gasleft();
```

```
// 가스 부족 검증 (63/64 규칙)
if (gasLeftAfter <= gasLimit / 63) {
assembly { invalid() }
}
return (success, result, gasLeftBefore - gasLeftAfter);
}
```

2. XTypes 라이브러리

2.1 핵심 데이터 구조

크로스체인 메시지:

```
struct Msg {
uint64 destChainId; // 목적지 체인 ID
uint64 shardId; // 샤드 ID (확인 레벨 포함)
uint64 offset; // 스트림 내 오프셋
address sender; // 송신자 주소
address to; // 수신자 주소
bytes data; // 호출 데이터
uint64 gasLimit; // 가스 한도
}
```

블록 헤더:

```
struct BlockHeader {
uint64 sourceChainId; // 소스 체인 ID
uint64 consensusChainId; // 합의 체인 ID
uint8 confLevel; // 확인 레벨
uint64 offset; // 블록 오프셋
uint64 sourceBlockHeight; // 소스 블록 높이
bytes32 sourceBlockHash; // 소스 블록 해시
}
```

제출 구조:

```
struct Submission {
bytes32 attestationRoot; // 증명 루트
uint64 validatorSetId; // 검증자 세트 ID
BlockHeader blockHeader; // 블록 헤더
Msg[] msgs; // 메시지 배열
bytes32[] proof; // 머클 증명
bool[] proofFlags; // 증명 플래그
SigTuple[] signatures; // 검증자 서명
}
```

2.2 검증자 및 서명

검증자 구조:

```
struct Validator {
address addr; // 검증자 주소
uint64 power; // 투표권
}
```

서명 튜플:

```
struct SigTuple {
address validatorAddr; // 검증자 주소
bytes signature; // 서명 (65바이트)
}
```

3. FeeOracleV2 컨트랙트

3.1 수수료 계산 메커니즘

수수료 파라미터:

```
struct FeeParams {
uint16 gasToken; // 가스 토큰 ID
uint32 baseGasLimit; // 기본 가스 한도
uint64 chainId; // 체인 ID
uint64 gasPrice; // 가스 가격
uint64 dataCostId; // 데이터 비용 ID
```

```
}
```

데이터 비용 파라미터:

```
struct DataCostParams {
uint16 gasToken; // 가스 토큰 ID
uint32 baseBytes; // 기본 바이트 수
uint64 id; // 데이터 비용 ID
uint64 gasPrice; // 가스 가격
uint64 gasPerByte; // 바이트당 가스
}
```

3.2 수수료 계산 공식

```
function feeFor(uint64 destChainId, bytes calldata data, uint64 gasLimit)
external view returns (uint256) {
FeeParams storage p = _feeParams[destChainId];
DataCostParams storage d = _dataCostParams[p.dataCostId];
// 네이티브 토큰으로 변환
uint256 _execGasPrice = p.gasPrice * tokenToNativeRate[p.gasToken] / CONVERSION_
uint256 _dataGasPrice = d.gasPrice * tokenToNativeRate[d.gasToken] / CONVERSION_
// 데이터 가스 계산 (바이트당 16 가스 가정)
uint256 dataGas = (d.baseBytes + data.length) * d.gasPerByte;
// 총 수수료 = 프로토콜 수수료 + 실행 비용 + 데이터 비용
```

```
return protocolFee +

(p.baseGasLimit + gasLimit) * _execGasPrice +

(dataGas * _dataGasPrice);

}
```

4. PortalRegistry 컨트랙트

4.1 포털 배포 관리

배포 정보 구조:

```
struct Deployment {

address addr; // 배포 주소

uint64 chainId; // 체인 ID

uint64 deployHeight; // 배포 높이

uint64 attestInterval; // 증명 간격

uint64 blockPeriodNs; // 블록 주기 (나노초)

uint64[] shards; // 지원 샤드

string name; // 체인 이름

}
```

4.2 포털 등록 과정

```
function _register(Deployment calldata dep) internal {
```

```
// 기본 검증

require(dep.addr != address(0), "PortalRegistry: zero addr");

require(dep.chainId > 0, "PortalRegistry: zero chain ID");

require(dep.attestInterval > 0, "PortalRegistry: zero interval");

// 샤드 유효성 검증

for (uint64 i = 0; i < dep.shards.length; i++) {

uint64 shard = dep.shards[i];

require(shard == uint8(shard) && ConfLevel.isValid(uint8(shard)),

"PortalRegistry: invalid shard");

}

// 등록 및 이벤트 발생

deployments[dep.chainId] = dep;

chainIds.push(dep.chainId);

emit PortalRegistered(

dep.chainId, dep.addr, dep.deployHeight,

dep.attestInterval, dep.blockPeriodNs, dep.shards, dep.name

);

}
```

5. XBlockMerkleProof 라이브러리

5.1 머클 증명 검증

검증 함수:

```
function verify(
bytes32 root,
XTypes.BlockHeader calldata blockHeader,
XTypes.Msg[] calldata msgs,
bytes32[] calldata msgProof,
bool[] calldata msgProofFlags
) internal pure returns (bool) {
// 메시지 머클 루트 계산
bytes32[] memory rootProof = new bytes32[](1);
rootProof[0] = MerkleProof.processMultiProofCalldata(
msgProof, msgProofFlags, _msgLeaves(msgs)
);
// 블록 헤더와 함께 최종 검증
return MerkleProof.verify(rootProof, root, _blockHeaderLeaf(blockHeader));
}
```

5.2 리프 해시 생성

도메인 분리 태그:

```
uint8 internal constant DST_XBLOCK_HEADER = 1; // 블록 헤더용
uint8 internal constant DST_XMSG = 2; // 메시지용
```

이중 해시 방식:

```
function _leafHash(uint8 dst, bytes memory leaf) private pure returns (bytes32)

// 이중 해시로 second preimage 공격 방지

return keccak256(bytes.concat(keccak256(abi.encodePacked(dst, leaf))));

}
```

6. 보안 메커니즘

6.1 재진입 방지

- `ReentrancyGuardUpgradeable` 사용
- `nonReentrant` 모디파이어 적용

6.2 일시 정지 기능

- 전체 또는 특정 체인별 일시 정지
- `xcall` 및 `xsubmit` 개별 제어

6.3 가스 안전성

- 63/64 가스 규칙 준수
- `excessivelySafeCall` 로 대용량 반환 데이터 방지

6.4 검증자 관리

- 쿼럼 기반 합의
- 검증자 세트 버전 관리
- 오래된 검증자 세트 차단

7. 이벤트 시스템

7.1 주요 이벤트

XMsg 이벤트:

```
event XMsg(
uint64 indexed destChainId,
uint64 indexed shardId,
uint64 indexed offset,
address sender,
address to,
bytes data,
uint64 gasLimit,
uint256 fees
);
```

XReceipt 이벤트:

```
event XReceipt(
uint64 indexed sourceChainId,
uint64 indexed shardId,
uint64 indexed offset,
uint256 gasUsed,
address relayer,
```

```
bool success,

bytes err

);
```

8. 업그레이드 가능성

8.1 프록시 패턴

- OpenZeppelin의 업그레이드 가능한 컨트랙트 사용
- 초기화 함수를 통한 상태 설정

8.2 스토리지 레이아웃

- `OmniPortalStorage` 에서 스토리지 변수 관리
- 업그레이드 시 스토리지 충돌 방지

결론

Omni의 Core 컨트랙트는 크로스체인 메시징의 핵심 기능을 안전하고 효율적으로 구현합니다. OmniPortal을 중심으로 한 아키텍처는 확장성, 보안성, 그리고 유지보수성을 모두 고려한 설계를 보여줍니다. 특히 머클 증명 기반의 검증 시스템과 쿼럼 합의 메커니즘은 크로스체인 통신의 신뢰성을 보장하는 핵심 요소입니다.